

**Arithmetic Expression Evaluator in C++**

**Test Case**

**Version 1.0**

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

## Revision History

| Date       | Version | Description                       | Author                                           |
|------------|---------|-----------------------------------|--------------------------------------------------|
| 04/26/2026 | 1.0     | Initial test case specification   | Team AEEDevelopment<br>(RG, Ayesha, Mahathi Sai) |
| 04/27/2026 | 1.0     | Test Case identifier              | Ayesha                                           |
| 04/27/2026 | 1.0     | Test items, Output Specifications | RG                                               |
| 04/28/2026 | 1.0     | Purpose, input specifications     | Mahathi Sai                                      |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

## Table of Contents

|                                    |                                     |
|------------------------------------|-------------------------------------|
| 1. Purpose                         | 4                                   |
| 2. Test case identifier            | 4                                   |
| 3. Test item                       | 6                                   |
| 4. Input specifications            | 6                                   |
| 5. Output specifications           | 8                                   |
| 6. Environmental needs             | 8                                   |
| 6.1.1 Hardware                     | 8                                   |
| 6.1.2 Software                     | 8                                   |
| 6.1.3 Other                        | <b>Error! Bookmark not defined.</b> |
| 7. Special procedural requirements | 9                                   |
| 8. Intercase dependencies          | 9                                   |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

## Test Case

### 1. Purpose

This document defines the test cases for the Arithmetic Expression Evaluator in C++ project. Its purpose is to systematically verify that the system satisfies the essential functional requirements defined in the Software Requirements Specification and behaves correctly for valid input, invalid input, boundary conditions, and error scenarios.

The test cases in this document are designed to support repeatable testing during implementation and later validation. They are based on the project description, the SRS, the software architecture, the implementation deliverable, and the project test expectations discussed in earlier team meetings.

### 2. Test case identifier

The following table lists the unique identifiers assigned to the test cases included in this document.

| Test Case ID | Description                            |
|--------------|----------------------------------------|
| TC01         | Verify simple addition                 |
| TC02         | Verify simple subtraction              |
| TC03         | Verify simple multiplication           |
| TC04         | Verify simple division                 |
| TC05         | Verify modulo operation                |
| TC06         | Verify exponentiation                  |
| TC07         | Verify operator precedence             |
| TC08         | Verify parentheses override precedence |
| TC09         | Verify nested parentheses              |
| TC10         | Verify unary minus                     |
| TC11         | Verify unary plus and minus together   |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

|      |                                              |
|------|----------------------------------------------|
| TC12 | Verify right associativity of exponentiation |
| TC13 | Verify left associativity of subtraction     |
| TC14 | Verify left associativity of division        |
| TC15 | Verify combined precedence with modulo       |
| TC16 | Verify longer valid expression               |
| TC17 | Verify whitespace handling                   |
| TC18 | Verify single integer input                  |
| TC19 | Verify negative result                       |
| TC20 | Verify divide-by-zero error                  |
| TC21 | Verify modulo-by-zero error                  |
| TC22 | Verify unsupported character error           |
| TC23 | Verify missing closing parenthesis           |
| TC24 | Verify malformed operator placement          |
| TC25 | Verify missing operator between values       |
| TC26 | Verify empty input                           |
| TC27 | Verify negative exponent rejection           |
| TC28 | Verify malformed operator sequence           |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

|      |                                  |
|------|----------------------------------|
| TC29 | Verify extra closing parenthesis |
| TC30 | Verify complex valid expression  |

### 3. Test item

The following system features and modules are covered by this test case specification:

- Arithmetic expression input handling
- Tokenization of valid expressions
- Parsing of expressions into a structured form
- Evaluation of binary arithmetic operators
- Evaluation of unary operators
- Parentheses handling
- Operator precedence and associativity
- Error handling for malformed expressions
- Error handling for invalid characters
- Division-by-zero and modulo-by-zero handling
- Command-line user interaction
- Initial robustness and boundary behavior

### 4. Input specifications

The following inputs are required to execute the test cases in this document.

The primary input is a user-entered arithmetic expression provided through the command-line interface.

Each test case uses a specific expression string as an input.

The relationships between inputs are as follows:

- Valid expressions must contain only supported integers, operators, whitespace, and parentheses.
- Operators must appear in valid positions according to the grammar of arithmetic expressions.
- Parentheses must be balanced for valid grouped expressions.
- Some invalid tests intentionally violate these rules to verify robustness and error handling.

The detailed test data are listed below.

| Test Case ID | Test Data |
|--------------|-----------|
| TC01         | 3 + 4     |
| TC02         | 10 - 6    |
| TC03         | 5 * 7     |
| TC04         | 20 / 5    |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

|      |                              |
|------|------------------------------|
| TC05 | $17 \% 5$                    |
| TC06 | $2 ** 3$                     |
| TC07 | $3 + 4 * 2$                  |
| TC08 | $(3 + 4) * 2$                |
| TC09 | $((2 + 3) * (4 - 1))$        |
| TC10 | $-(2 * (-3))$                |
| TC11 | $-(+1) + (+2)$               |
| TC12 | $2 ** 3 ** 2$                |
| TC13 | $10 - 3 - 2$                 |
| TC14 | $100 / 5 / 4$                |
| TC15 | $4 * (3 + 2) \% 7 - 1$       |
| TC16 | $8 + 2 * 3 - 4 / 2 + 5 \% 3$ |
| TC17 | $12 + 8 * 2$                 |
| TC18 | 42                           |
| TC19 | $3 - 10$                     |
| TC20 | $4 / 0$                      |
| TC21 | $10 \% 0$                    |

|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

|      |                                   |
|------|-----------------------------------|
| TC22 | 7 & 3                             |
| TC23 | 2 * (4 + 3 - 1                    |
| TC24 | * 5 + 2                           |
| TC25 | 5 (2 + 3)                         |
| TC26 | empty input                       |
| TC27 | 2 ** -3                           |
| TC28 | 5 + * 2                           |
| TC29 | (3 + 2))                          |
| TC30 | ((5 * 2) - ((3 / 1) + ((4 % 3)))) |

## 5. Output specifications

The output for each test case must be either the correct evaluated result for valid expressions or a clear error message for invalid expressions and runtime errors.

The expected outputs are listed below.

## 6. Environmental needs

### 6.1.1 Hardware

No special hardware is required beyond a standard computer capable of compiling and running a C++ command-line application.

### 6.1.2 Software

The software environment required for these test cases includes the project source code, build configuration, command-line interface, and C++ build environment described in the repository and implementation materials.



|                                        |                  |
|----------------------------------------|------------------|
| Arithmetic Expression Evaluator in C++ | Version: 1.0     |
| Test Case                              | Date: 10/05/2026 |

## 7. Special procedural requirements

The program should be built successfully before the test cases are executed. Each test case should be run independently by entering the specified expression through the command-line interface.

For consistency, actual results and pass/fail status should be recorded immediately after each execution. If the implementation changes, the same test cases should be rerun, so results remain current and repeatable.

## 8. Intercase dependencies

The program should be built successfully before the test cases are executed. Each test case should be run independently by entering the specified expression through the command-line interface.

For consistency, actual results and pass/fail status should be recorded immediately after each execution. If the implementation changes, the same test cases should be rerun, so results remain current and repeatable.